

Sesión 10 – Monitores: resolución de problemas de sincronización



Contexto de la sesión

Esta sesión sigue la dinámica establecida en la sesión 7. No hay ficheros esqueleto, no hay tests automáticos y no se escribe una sola línea de Java. El trabajo es íntegramente conceptual: modelar dos problemas de sincronización usando monitores y expresar la solución en pseudocódigo estructurado.

El dominio de los problemas es el mismo sistema de gestión de inventario del almacén con el que se ha trabajado desde la sesión 1. Esa continuidad no es casual: permite concentrar el esfuerzo cognitivo en el mecanismo de sincronización, no en entender el contexto del negocio.

La sesión 7 abordó estos mismos objetivos con semáforos. Ahora se resuelven problemas equivalentes en complejidad, pero con una herramienta distinta: el monitor. La comparación implícita entre ambas sesiones es deliberada y muy formativa. El alumno comprobará que el monitor internaliza la exclusión mutua que en los semáforos había que gestionar manualmente, pero que las variables condición exigen un razonamiento propio que no tiene equivalente directo en el modelo de semáforos.

Los problemas tienen la misma naturaleza y nivel de dificultad que los ejercicios de monitores del examen de evaluación de la asignatura. Conviene abordarlos con la misma seriedad y rigor que se aplicaría en ese contexto.

Objetivos de aprendizaje

Al finalizar esta sesión el alumno será capaz de:

- Identificar los recursos compartidos y las condiciones de espera en un enunciado en prosa.
- Encapsular el estado compartido y las operaciones sobre él dentro de la estructura de un monitor.

- Diseñar variables condición (*condition variables*) que representen las condiciones de espera de cada tipo de proceso.
- Utilizar correctamente las operaciones `delay(c)`, `resume(c)` y `empty(c)` bajo la semántica de retorno forzado (*immediate resumption*).
- Aprovechar la función `empty(c)` para tomar decisiones condicionales sobre qué variable condición señalar.
- Escribir pseudocódigo de procesos y procedimientos de acceso de un monitor libre de interbloqueo (*deadlock*) y de inanición (*starvation*).
- Comparar, de forma cualitativa, la expresividad del monitor frente a la del semáforo para el mismo tipo de problema.

Temporización orientativa

Bloque	Tiempo	Actividad
Análisis	30 min	Leer los dos enunciados, identificar procesos, estado encapsulado y condiciones de espera
Resolución	60 min	Declarar variables condición, escribir los procedimientos de acceso del monitor y el pseudocódigo de cada proceso
Revisión	30 min	Buscar escenarios problemáticos, completar el argumento de corrección, exportar el PDF

El análisis previo es la parte más importante y la más descuidada. Antes de escribir una sola línea de pseudocódigo, el alumno debe tener claro: qué estado encapsula el monitor, qué procedimientos de acceso expone y cuál es la condición de espera asociada a cada variable condición.

Herramienta de resolución: monitores

Un monitor es un módulo de sincronización que encapsula un estado compartido y los procedimientos que actúan sobre él, garantizando que en cada instante solo un proceso puede ejecutar un procedimiento de acceso del monitor. Esta propiedad — la exclusión

mutua implícita — es la diferencia fundamental respecto a los semáforos, donde la exclusión mutua debía gestionarse explícitamente con operaciones `wait / signal` sobre un semáforo binario.

Estructura de un monitor

Un monitor consta de tres elementos:

- **Variables internas:** el estado compartido que el monitor protege. Ningún proceso externo puede acceder directamente a estas variables; solo son visibles dentro del monitor.
- **Procedimientos de acceso** (*entry procedures*): los procedimientos que los procesos externos invocan para interactuar con el estado compartido. Son los únicos puntos de entrada al monitor y se declaran explícitamente en la cabecera del monitor mediante una línea `procedimientos de acceso:` seguida de sus nombres. La ejecución de cualquier procedimiento de acceso es mutuamente excluyente: si un proceso está ejecutando uno, cualquier otro proceso que intente invocar uno (el mismo u otro distinto) queda bloqueado hasta que el primero termine o se suspenda explícitamente. Junto a los procedimientos de acceso, el monitor puede contener procedimientos privados, que son auxiliares internos invocables solo desde otros procedimientos del propio monitor.
- **Variables condición:** mecanismos de espera que permiten a un proceso que ya ha entrado al monitor suspenderse voluntariamente hasta que se cumpla una condición determinada, cediendo temporalmente la exclusión mutua para que otros procesos puedan progresar.

Variables condición

Una variable condición `c` soporta tres operaciones:

- `delay(c)` — suspende al proceso que la invoca y libera la exclusión mutua del monitor, permitiendo que otro proceso entre. Cuando el proceso sea despertado, volverá a adquirir la exclusión mutua antes de continuar ejecutándose.
- `resume(c)` — si hay algún proceso suspendido en `c`, despierta exactamente uno **y el proceso que ejecutó el `resume` sale inmediatamente del monitor** (retorno forzado). Si no hay ningún proceso suspendido, la operación no tiene efecto y el proceso que la invocó continúa con normalidad.

- `empty(c)` — devuelve `true` si no hay ningún proceso suspendido en `c`; `false` en caso contrario. Es una consulta pura: no modifica el estado ni bloquea al proceso.

A diferencia de un semáforo, donde un `signal` incrementa un contador y su efecto persiste aunque nadie esté esperando, en un monitor el `resume` sobre una variable condición no tiene efecto si nadie está suspendido en ella. Esta diferencia es esencial y una fuente frecuente de errores.

Semántica de retorno forzado (*immediate resumption*)

La semántica que se utiliza en esta asignatura para la operación `resume` es la de **retorno forzado** (*immediate resumption*): cuando un proceso `P` ejecuta `resume(c)` y hay un proceso `Q` suspendido en `c`, `Q` reanuda su ejecución inmediatamente dentro del monitor y `P` abandona el monitor de forma forzada. Es decir, `P` no continúa ejecutando instrucciones después del `resume`: su ejecución dentro del procedimiento de acceso termina en ese punto.

Esta semántica tiene dos consecuencias prácticas fundamentales:

1. **La condición que motivó el `resume` sigue siendo cierta** cuando `Q` reanuda su ejecución, porque ningún otro proceso ha tenido oportunidad de modificar el estado del monitor entre el `resume` y la reanudación de `Q`. Esto significa que `Q` no necesita volver a comprobar la condición: un simple `si` antes del `delay` es suficiente, y no es necesario usar un bucle `mientras`.
2. **El `resume` debe ser la última operación significativa** dentro de la rama de ejecución que lo contiene. Cualquier instrucción escrita después del `resume` en la misma rama no será ejecutada por el proceso que lo invocó (porque ha sido forzado a salir). Si el proceso necesita realizar acciones adicionales después de señalar, debe completarlas antes de ejecutar el `resume`.

La función `empty(c)` es especialmente útil en esta semántica. Dado que `resume` fuerza la salida del monitor, conviene comprobar con `empty(c)` si merece la pena señalar antes de hacerlo. Esto permite al proceso tomar decisiones sobre qué variable condición señalar (o si debe señalar alguna) sin verse forzado a abandonar el monitor innecesariamente. Un patrón habitual es:

```
procedimiento ejemplo() {
    // modificar el estado del monitor
```

```
if (!empty(condicionA)) {
    resume(condicionA)      // sale del monitor aqui
} else if (!empty(condicionB)) {
    resume(condicionB)      // sale del monitor aqui
}
// si llega aqui, no habia nadie esperando
}
```

Formato del pseudocódigo

Declaración del monitor

El monitor se declara con su nombre, la lista de procedimientos de acceso en la cabecera, sus variables internas (con tipo y valor inicial), sus variables condición y el cuerpo de cada procedimiento:

```
monitor NombreMonitor {
    // variables internas
    int contador = 0
    boolean activo = false

    // variables condicion
    condicion puedeAvanzar    // procesos esperando a que se cumpla cierta

    procedimientos de acceso: entrar, salir

    procedimiento entrar() {
        // cuerpo del procedimiento
    }

    procedimiento salir() {
        // cuerpo del procedimiento
    }
}
```

La línea procedimientos de acceso: enumera los procedimientos que los procesos externos pueden invocar. Todo procedimiento que no aparezca en esa lista se considera privado y solo es invocable desde otros procedimientos del mismo monitor.

Incluir siempre un comentario que explique qué condición de espera representa cada variable condición.

Estructura de proceso

Los procesos son externos al monitor e invocan sus procedimientos de acceso:

```
proceso NombreProceso(parametros opcionales) {  
    repetir indefinidamente {  
        // trabajo previo (fuera del monitor)  
        NombreMonitor.nombreProcedimiento(argumentos)  
        // trabajo posterior (fuera del monitor)  
    }  
}
```

El proceso nunca accede directamente a las variables internas del monitor. Toda interacción con el estado compartido se realiza a través de los procedimientos de acceso del monitor.

Modularización de procedimientos

Cuando un procedimiento de acceso del monitor es complejo, puede descomponerse en procedimientos privados internos al monitor. Estos procedimientos no son invocables desde fuera del monitor; son mecanismos de organización interna. Comparar las dos versiones siguientes:

Sin modularizar — toda la lógica en un único procedimiento:

```
procedimiento procesarPeticion(int id) {  
    // comprobar si se puede procesar; si no, esperar  
    // actualizar el estado interno  
    // despertar a quien corresponda  
}
```

Modularizado — cada fase con nombre propio:

```
procedimiento privado esperarTurno() {  
    if (!puedeProcesar) {  
        delay(puedeAvanzar)  
    }  
}
```

```
procedimiento privado actualizarEstado(int id) {  
    contador = contador + 1  
}  
  
procedimiento procesarPeticion(int id) {  
    esperarTurno()  
    actualizarEstado(id)  
    if (!empty(puedeAvanzar)) {  
        resume(puedeAvanzar)  
    }  
}
```

La segunda versión es más larga, pero cada procedimiento privado es verificable de forma aislada y el procedimiento de acceso se lee como una secuencia de acciones con nombre.

Operaciones sobre variables condición

Se admiten las notaciones equivalentes; usar la misma de forma consistente en toda la solución:

Notación procedural	Notación objetos
delay(c)	c.delay()
resume(c)	c.resume()
empty(c)	c.empty()

No se admite mezclar las notaciones dentro del mismo ejercicio.

Lo que no debe aparecer en la solución

- Ninguna referencia a Java ni a sus APIs: ni `Lock`, ni `Condition`, ni `synchronized`, ni `wait()`, ni `notify()`, ni `notifyAll()`.
- Uso de `signal` / `signalAll` en lugar de `resume`. La nomenclatura de la asignatura para las operaciones sobre variables condición es `delay`, `resume` y `empty`.
- Implementación de estructuras de datos. Si el problema requiere una cola, un buffer u otra estructura auxiliar, puede nombrarse como recurso abstracto y

utilizarse mediante funciones descriptivas (`depositar(buffer, elemento)` , `retirar(buffer)` , `estaVacio(buffer)` , etc.) sin desarrollar su implementación interna, ya que esta es competencia de otras asignaturas de la titulación. Sin embargo, el alumno **debe incluir en su solución una descripción del comportamiento de cada función** que utilice sobre la estructura de datos: qué recibe, qué devuelve y qué efecto tiene sobre el estado de la estructura.

- Gestión de excepciones, manejo de interrupciones ni ningún otro detalle de plataforma.
- Variables que no sean necesarias para modelar la sincronización.
- Caracteres especiales del idioma español (tildes, eñes) dentro del pseudocódigo. El pseudocódigo utiliza una nomenclatura cercana a Java: `int` , `boolean` , `long` para tipos; `if / else` para condicionales; `true / false` para valores lógicos.

Constantes simbólicas

En la solución, todos los valores numéricos con significado de negocio deben definirse como constantes simbólicas y referenciarse por nombre, nunca como literales en el código de los procesos. Por ejemplo, un límite de capacidad no debe aparecer como `10` en el pseudocódigo sino como `MAX_CAPACIDAD` , y un número máximo de operaciones no como `50` sino como `MAX_OPERACIONES` . Este requisito no es accesorio: las constantes simbólicas hacen el pseudocódigo verificable (se puede comprobar que el valor inicial de una variable coincide con la constante declarada) y constituyen un criterio de evaluación en el examen.

Todas las constantes del enunciado — cantidades de recursos, tamaños de lote y números de procesos — siguen una convención uniforme: nombres en mayúsculas, palabras separadas por guion bajo y prefijo `N_` para cardinalidades. Mantener esta convención en la solución es parte del requisito anterior.

Estructura de la solución

Cada ejercicio debe resolverse siguiendo esta estructura, que también es la que se valorará en el examen:

1. Análisis del problema

Identificar, en prosa, los siguientes elementos antes de escribir una sola línea de pseudocódigo:

- **Procesos:** cuántos tipos hay, cuántas instancias de cada tipo y cuál es su ciclo de trabajo.
- **Estado compartido:** qué datos deben encapsularse dentro del monitor porque son accedidos por varios procesos.
- **Procedimientos de acceso del monitor:** qué procedimientos de acceso expone el monitor a los procesos externos, y cuáles son sus precondiciones y postcondiciones.
- **Condiciones de espera:** para cada situación en la que un proceso debe suspenderse, qué condición concreta debe cumplirse para que pueda continuar. Cada condición distinta requiere una variable condición.

Este análisis es imprescindible. Una solución sin análisis previo, aunque sea correcta, no demuestra que se comprende el problema.

2. Declaración del monitor

Para cada variable interna: nombre descriptivo, tipo, valor inicial y justificación del valor. Para cada variable condición: nombre descriptivo y descripción de la condición de espera que representa. La justificación no puede ser trivial; debe explicar qué representa ese valor o esa condición en el contexto del problema.

3. Pseudocódigo del monitor y de los procesos

Pseudocódigo completo de los procedimientos de acceso del monitor, los procedimientos privados auxiliares si los hubiera, y de cada tipo de proceso. Los procedimientos del monitor se presentan dentro de la estructura `monitor` `NombreMonitor { ... }`. Los procesos se presentan fuera del monitor. El código debe ser suficientemente preciso como para que no haya ambigüedad sobre el orden de las operaciones `delay()` y `resume()`.

4. Argumento de corrección

Demostrar, de forma razonada pero concisa, que la solución satisface las tres propiedades siguientes. Cada propiedad se evalúa de forma independiente en los

criterios ponderados, por lo que conviene presentarlas en subsecciones claramente diferenciadas:

- **Corrección funcional:** los invariantes del enunciado se mantienen en cualquier intercalación posible. Es la propiedad de seguridad (*safety*) del sistema.
- **Ausencia de interbloqueo** (*deadlock freedom*): no existe ningún escenario en el que todos los procesos queden bloqueados indefinidamente esperando entre sí. Argumentar identificando todas las vías por las que un proceso suspendido en `delay(c)` puede ser despertado y comprobando que al menos una de ellas está siempre activa mientras el sistema progresa.
- **Ausencia de inanición** (*starvation freedom*): todo proceso que desee progresar acabará haciéndolo en algún momento, asumiendo que el planificador es equitativo (*fair*). Si la solución no garantiza esta propiedad en todos los casos, identificar explícitamente los escenarios problemáticos y los supuestos necesarios para que la solución siga siendo aceptable.

Las dos primeras propiedades son condición necesaria para la evaluación del ejercicio, como se detalla en la sección de criterios de evaluación. La tercera admite matices razonados.

Problema 1 — Equipamiento de la zona de carga

Enunciado

El almacén dispone de una **zona de carga** donde los operarios manipulan mercancías con la ayuda de dos tipos de equipamiento: **carretillas elevadoras** y **transpaletas**. Del primer tipo hay `N_CARRETILLAS` unidades disponibles y del segundo hay `N_TRANSPALETAS` unidades. Ambos tipos de equipamiento son compartidos y reutilizables: cuando un operario termina su tarea, devuelve el equipamiento al pool de la zona de carga para que otros operarios lo utilicen.

Los operarios se clasifican en tres tipos según la tarea que van a realizar:

- **OperarioDescarga** (hay `N_OPERARIOS_DESCARGA` procesos): cada operario de descarga necesita **una carretilla elevadora** para descargar palets de los camiones. Ejecuta un ciclo de trabajo indefinido en el que solicita una carretilla, realiza la descarga y devuelve la carretilla.

- **OperarioReposicion** (hay `N_OPERARIOS_REPOSICION` procesos): cada operario de reposición necesita **una transpaleta** para mover productos desde la zona de recepción hasta las estanterías. Ejecuta un ciclo de trabajo indefinido en el que solicita una transpaleta, realiza la reposición y devuelve la transpaleta.
- **OperarioCargaPesada** (hay `N_OPERARIOS_CARGA_PESADA` procesos): cada operario de carga pesada necesita **una carretilla elevadora y una transpaleta simultáneamente** para mover mercancías sobredimensionadas que requieren ambos equipos coordinados. Ejecuta un ciclo de trabajo indefinido en el que solicita ambos equipos a la vez, realiza la operación y devuelve ambos.

Las restricciones del sistema son las siguientes:

- **Política de asignación con prioridad para la carga pesada.** Un operario de descarga o de reposición podrá obtener una unidad del equipamiento que solicita si y solo si se cumplen simultáneamente dos condiciones: primero, que haya al menos una unidad libre de ese tipo; y segundo, que no haya ningún operario de carga pesada esperando en ese momento. Un operario de carga pesada podrá obtener equipamiento si y solo si hay al menos una carretilla libre y al menos una transpaleta libre de forma simultánea. Esto significa que, mientras haya algún operario de carga pesada esperando, las unidades libres permanecen reservadas implícitamente para él aunque los operarios de descarga o reposición también las necesiten.
- **Política de desbloqueo con prioridad para la carga pesada.** Cuando un operario devuelve equipamiento, pueden desbloquearse uno o más operarios que estuvieran esperando. La política de prioridad debe respetarse también en ese momento: si hay operarios de carga pesada esperando y el equipamiento devuelto permite atender su solicitud (es decir, hay al menos una carretilla y una transpaleta libres tras la devolución), deben desbloquearse ellos antes que los operarios de descarga o de reposición. Si la devolución no permite atender a ningún operario de carga pesada en espera, los operarios de descarga y reposición siguen sin poder obtener equipamiento mientras siga habiendo operarios de carga pesada en espera.
- El número de procesos de cada tipo es arbitrario y se desconoce en tiempo de diseño.
- Los procesos de los tres tipos repiten su ciclo de trabajo indefinidamente.

Problema 2 — Estación de empaquetado por lotes

Enunciado

El almacén cuenta con una **estación de empaquetado** donde se preparan los pedidos para su expedición. La estación opera con un modelo de trabajo **por lotes**: cada lote de empaquetado requiere la presencia simultánea de exactamente **un supervisor de calidad** y exactamente **N_OPERARIOS_LOTE operarios de empaquetado**. Un lote no puede comenzar hasta que se haya formado el equipo completo, y ningún miembro del equipo puede avanzar de forma individual.

En la estación trabajan dos tipos de procesos:

- **SupervisorCalidad** (hay N_SUPERVISORES procesos): cada supervisor ejecuta un ciclo de trabajo indefinido en el que se presenta en la estación para supervisar un lote. Cuando llega a la estación, llama a la operación de sincronización. Si no hay al menos N_OPERARIOS_LOTE operarios bloqueados en la estación, el supervisor **debe esperar** hasta que se complete el equipo. Una vez formado el equipo, el supervisor realiza la supervisión del lote y después vuelve a presentarse para el siguiente.
- **OperarioEmpaquetado** (hay N_OPERARIOS_EMPAQUETADO procesos): cada operario ejecuta un ciclo de trabajo indefinido en el que se presenta en la estación para trabajar en un lote. Cuando llega, llama a la operación de sincronización. Si no hay al menos un supervisor y $N_OPERARIOS_LOTE - 1$ operarios más (aparte de él mismo) bloqueados en la estación, el operario **debe esperar** hasta que se complete el equipo. Una vez formado el equipo, el operario realiza el empaquetado del lote y después vuelve a presentarse para el siguiente.

La sincronización funciona de la siguiente manera:

- Si un supervisor llama a la operación de sincronización y hay al menos N_OPERARIOS_LOTE operarios bloqueados en dicha operación, el supervisor no se bloquea y además deben desbloquearse exactamente N_OPERARIOS_LOTE operarios.
- Si un operario llama a la operación de sincronización y hay al menos 1 supervisor y $N_OPERARIOS_LOTE - 1$ operarios (aparte de él mismo) bloqueados en dicha operación, el operario no se bloquea y además deben desbloquearse exactamente 1 supervisor y $N_OPERARIOS_LOTE - 1$ operarios.

- En cualquier otro caso, el proceso que llama a la operación de sincronización se bloquea hasta que se cumplan las condiciones anteriores.
- Cada lote consume exactamente 1 supervisor y `N_OPERARIOS_LOTE` operarios del conjunto de procesos esperando. Los procesos sobrantes (supervisores u operarios que no entraron en el lote) permanecen bloqueados esperando al siguiente lote.

Las restricciones del sistema son las siguientes:

- El equipo de un lote se forma de manera atómica: o se desbloquean todos los miembros del lote (1 supervisor + `N_OPERARIOS_LOTE` operarios) o no se desbloquea ninguno.
- No puede ocurrir que un operario trabaje en un lote sin supervisor, ni que un supervisor supervise un lote con menos de `N_OPERARIOS_LOTE` operarios.
- Los procesos que se desbloquean como parte de un lote pueden proceder a ejecutar su trabajo (el supervisor supervisa, los operarios empaquetan) sin necesidad de sincronización adicional dentro de ese lote. La sincronización es exclusivamente de formación del equipo.
- El número de supervisores `N_SUPERVISORES` y de operarios `N_OPERARIOS_EMPAQUETADO` es arbitrario. Puede haber más supervisores que operarios o viceversa. El sistema debe funcionar correctamente en cualquier configuración.
- Los procesos de ambos tipos repiten su ciclo de trabajo indefinidamente.

Entrega

La entrega consta de dos ficheros comprimidos en un único archivo ZIP, que es lo que debe subirse a la tarea correspondiente en Platea. No se aceptarán ficheros sueltos ni enviados por correo electrónico.

Fichero 1 – Solución en Markdown

Nombre del fichero: `solucion-<apellido>-<nombre>.md`

Un único fichero Markdown que contenga la solución completa de los dos problemas, siguiendo la estructura descrita en la sección anterior (análisis, declaración del monitor, pseudocódigo y argumento de corrección para cada problema). El fichero debe

renderizarse correctamente en cualquier visor de Markdown estándar (GitHub, VS Code, Typora, Obsidian). Los bloques de pseudocódigo deben delimitarse con triple acento grave y la etiqueta `pseudocode` :

```
```pseudocode
monitor EjemploFormato {
 int valor = VALOR_INICIAL
 condicion puedeAvanzar

 procedimientos de acceso: incrementar

 procedimiento incrementar() {
 valor = valor + 1
 if (!empty(puedeAvanzar)) {
 resume(puedeAvanzar)
 }
 }
}
```
```

Fichero 2 – PDF generado desde el Markdown

Nombre del fichero: `solucion-<apellido>-<nombre>.pdf`

El mismo contenido que el fichero Markdown, exportado a PDF. Se admite cualquier método de conversión que produzca un resultado legible: exportación directa desde VS Code (extensión Markdown PDF), desde Typora, desde Pandoc, o simplemente abriendo el Markdown en cualquier visor que lo renderice (VS Code, Obsidian, un navegador con extensión de Markdown) y usando la función **Imprimir → Guardar como PDF** del sistema operativo. Esta última opción es la más sencilla y no requiere instalar herramientas adicionales.

El PDF no debe contener anotaciones ni contenido adicional respecto al Markdown. Si se desea adjuntar esquemas o diagramas elaborados durante el análisis, deben incluirse en el propio Markdown como imágenes embebidas o como secciones de texto adicionales, no en un documento separado.

Empaquetado en ZIP

Los dos ficheros anteriores deben comprimirse en un único archivo ZIP con la siguiente estructura interna, sin carpetas intermedias:

```
solucion-<apellido>-<nombre>.zip
├─ solucion-<apellido>-<nombre>.md
└─ solucion-<apellido>-<nombre>.pdf
```

Desde la línea de comandos, estando en el directorio donde están los dos ficheros:

```
zip solucion-apellido-nombre.zip \
    solucion-apellido-nombre.md \
    solucion-apellido-nombre.pdf
```

Para generar el PDF desde el Markdown con Pandoc, disponible en los laboratorios de la EPS:

```
pandoc solucion-apellido-nombre.md \
    -o solucion-apellido-nombre.pdf \
    --pdf-engine=xelatex \
    -V geometry:margin=2.5cm \
    -V fontsize=11pt \
    -V lang=es
```

Si `xelatex` no está disponible en el equipo, usar `--pdf-engine=pdflatex` como alternativa.

Criterios de evaluación

Los ejercicios se evalúan con los mismos criterios que se aplicarían en el examen de la asignatura.

Condiciones necesarias

Los siguientes criterios son **condición necesaria** para la evaluación del ejercicio. Si alguno de ellos no se satisface, el ejercicio se califica con un 0 independientemente de la calidad del resto de la solución:

- **Corrección funcional del pseudocódigo.** El pseudocódigo debe satisfacer todos los invariantes del enunciado — es decir, las propiedades de seguridad (*safety*) que el problema exige. Una solución que no garantiza sus invariantes en cualquier intercalación posible de los procesos no es una solución al problema planteado.
- **Ausencia de interbloqueo** (*deadlock freedom*). No debe existir ningún escenario en el que todos los procesos queden bloqueados indefinidamente esperando entre sí.
- **Resolución exclusiva con monitores.** Una solución que utilice semáforos dentro del monitor o fuera de él, en lugar de las operaciones `delay` / `resume` / `empty` sobre variables condición, se considera que no resuelve el problema planteado en los términos del enunciado.

Un ejercicio que supera las tres condiciones necesarias se evalúa a continuación según los criterios ponderados.

Criterios ponderados

| Criterio | Peso | Descripción |
|--|------|--|
| Análisis del problema | 20 % | Se identifican correctamente los procesos, estado compartido y condiciones de espera |
| Declaración del monitor | 25 % | Variables internas y condición correctamente declaradas, inicializadas y justificadas |
| Calidad del pseudocódigo | 25 % | Pseudocódigo preciso, sin ambigüedades, modularizado, con constantes simbólicas y uso correcto de <code>delay</code> / <code>resume</code> / <code>empty</code> bajo retorno forzado |
| Razonamiento de corrección funcional | 15 % | Se argumenta que los invariantes del enunciado se mantienen en cualquier intercalación posible |
| Razonamiento de ausencia de interbloqueo | 10 % | Se argumenta que no existe ningún escenario de bloqueo circular, identificando las vías de desbloqueo de cada <code>delay</code> |
| Razonamiento de ausencia de inanición | 5 % | Se argumenta sobre la vivacidad de la solución y se identifican sus límites si los hay |

| Criterio | Peso | Descripción |
|----------------|----------------|---|
| Penalizaciones | hasta
-50 % | Presencia de código Java o referencias a APIs concretas |

Las tres primeras propiedades (análisis, declaración y calidad del pseudocódigo) evalúan el diseño de la solución. Las tres siguientes (razonamientos de corrección funcional, ausencia de interbloqueo y ausencia de inanición) evalúan la capacidad del alumno de justificar por qué su solución funciona. Una solución que sea funcionalmente correcta pero no incluya los razonamientos de corrección obtendrá como máximo el 70 % de la nota del ejercicio.

Nótese que la corrección funcional y la ausencia de interbloqueo aparecen simultáneamente como condiciones necesarias (evaluación del pseudocódigo) y como criterios ponderados (evaluación del razonamiento del alumno sobre ese pseudocódigo). No es duplicidad: lo primero comprueba que el pseudocódigo cumple la propiedad; lo segundo comprueba que el alumno sabe justificar por qué la cumple.

Recomendaciones de trabajo

Antes de escribir el pseudocódigo, identificar los **procedimientos de acceso del monitor** a partir de las acciones que los procesos necesitan realizar sobre el estado compartido. Un buen criterio: si dos tipos de proceso realizan acciones distintas sobre el mismo recurso, probablemente cada acción corresponde a un procedimiento de acceso del monitor.

Para cada procedimiento de acceso que pueda bloquear al proceso, definir una variable condición asociada e identificar cuál es la condición booleana que se evaluará antes del `delay`. Si dos tipos de proceso esperan por condiciones distintas, necesitan variables condición distintas.

Tener siempre presente que `resume(c)` fuerza al proceso que lo invoca a salir del monitor. Las consecuencias prácticas de este retorno forzado son dos: primero, toda instrucción posterior al `resume` en la misma rama de ejecución no se ejecutará; segundo, si el proceso necesita realizar varias acciones después de modificar el estado (por ejemplo, actualizar un contador y luego señalar), debe completar todas las

acciones *antes* del `resume` . Un patrón frecuente es usar `empty(c)` para decidir si señalar y a quién:

```
if (!empty(condicionA)) {  
    resume(condicionA)          // retorno forzado: sale del monitor  
} else if (!empty(condicionB)) {  
    resume(condicionB)          // retorno forzado: sale del monitor  
}  
// si llega aqui, nadie estaba esperando: el proceso sale normalmente
```

Para verificar la ausencia de interbloqueo en un monitor, comprobar que todo procedimiento de acceso que ejecuta `delay(c)` puede ser desbloqueado por otro procedimiento de acceso del mismo monitor que ejecute `resume(c)` , y que ese otro procedimiento es invocable por un proceso que no esté bloqueado dentro del monitor. Si todos los procesos que podrían ejecutar el `resume` necesitan primero entrar al monitor, y todos están suspendidos dentro de él, hay interbloqueo.

Análisis de inanición por problema

La ausencia de inanición es la propiedad más sutil de las tres y requiere un análisis adaptado al problema concreto. En ambos problemas de esta sesión existen situaciones donde la vivacidad no es trivial:

En el problema 1, la prioridad de los operarios de carga pesada sobre los ordinarios introduce un riesgo evidente: bajo una carga alta de operarios de carga pesada, los operarios de descarga y de reposición pueden quedar sin acceso al equipamiento durante periodos arbitrariamente largos, incluso si las unidades que solicitan están libres. El alumno debe analizar explícitamente este riesgo, identificar los supuestos bajo los cuales la solución es aceptable (por ejemplo, que la población de operarios de carga pesada sea finita y que cada ciclo de trabajo termine) y discutir si la prioridad estricta es compatible con la ausencia de inanición a largo plazo.

En el problema 2, el riesgo proviene de la política de señalización sobre las variables condición. Si un supervisor concreto lleva esperando mientras llegan y son servidos otros supervisores posteriores, o si el desbloqueo de los `N_OPERARIOS_LOTE` operarios favorece sistemáticamente a los últimos en llegar frente a los primeros, habrá procesos que nunca entren en un lote. El alumno debe analizar bajo qué política de gestión de las variables condición (FIFO, por ejemplo) la solución queda libre de inanición, e indicar qué supuestos son necesarios sobre el comportamiento de `resume` cuando hay varios procesos suspendidos en la misma variable condición.

Bibliografía de referencia

Los fundamentos necesarios para resolver estos problemas están cubiertos en los siguientes capítulos:

- **Andrews, G.R.** (2000). *Foundations of Multithreaded, Parallel, and Distributed Programming*. Capítulo 5: monitores, variables condición y disciplinas de señalización. La sección 5.1 introduce la estructura del monitor y las operaciones `delay` / `resume` / `empty` ; las secciones 5.2–5.4 desarrollan problemas clásicos resueltos con monitores, incluyendo patrones de señalización encadenada que resultan útiles cuando un único evento debe desbloquear a varios procesos. Disponible en la biblioteca de la EPS.
- **Ben-Ari, M.** (2006). *Principles of Concurrent and Distributed Programming*, 2.^a ed. Capítulo 7 (monitores): presenta la semántica de reanudación inmediata y la compara con la de señalización y continuación. Los problemas resueltos del capítulo son un buen complemento a los de esta sesión. Disponible en O'Reilly Learning con acceso UJA.
- **Hoare, C.A.R.** (1974). *Monitors: An Operating System Structuring Concept*. *Communications of the ACM*, 17(10), 549–557. El artículo original que define el concepto de monitor y la semántica de señalización con reanudación inmediata del proceso señalizado.
- **Brinch Hansen, P.** (1975). *The Programming Language Concurrent Pascal*. *IEEE Transactions on Software Engineering*, SE-1(2), 199–207. La primera implementación de monitores como constructo de un lenguaje de programación, con la restricción de que el `resume` debe ser la última sentencia del procedimiento de acceso.